Introduction

📄 Document Signature App — Python Stack

🧠 Introduction

The **Document Signature App** is a secure, scalable web application that enables users to **upload PDFs, place digital signatures, share signing links, and generate signed documents with audit trails**.

This Python-based implementation focuses on **backend reliability, security, and data integrity**, making it ideal for **legal, HR, finance, and enterprise workflows** where traceability and compliance matter.

By using Python frameworks and libraries, this project mirrors how **real-world SaaS products** handle document signing while remaining beginner-friendly and production-ready.

🎯 Aim of the Project

🎯 Aim of the Project

The aim of this project is to:

- Build a **DocuSign-like digital signature system** using Python
- Learn **secure backend architecture** with authentication & authorization
- Implement **PDF manipulation and digital signature embedding**
- Design **audit-ready workflows**
- Prepare a **job-ready, enterprise-grade portfolio project**

This project goes beyond CRUD by modeling **real business logic and security constraints**.

🧩 Problems This App Solves

🧩 Problems This App Solves

- Manual document signing delays workflows
- Paper-based processes lack traceability
- No audit logs for compliance verification
- High risk of document tampering
- No visibility into document status

This app solves these by offering **secure uploads, controlled access, immutable signed PDFs, and audit trails**.

🤵 Real-World Use Cases

🧑‍💼 Real-World Use Cases

1️⃣ HR & Recruitment

- Offer letters
- NDA agreements
- Policy acknowledgements

2️⃣ Legal & Compliance

- Contract signing
- Consent documents
- Regulatory approvals

3️⃣ Freelancers & Agencies

- Client agreements
- Proposal approvals
- Invoice authorizations

4️⃣ Education & Institutions

- Admission forms
- Certificates
- Consent forms

5️⃣ Healthcare & Finance

- Authorization forms

- Secure document approvals
- Compliance-based workflows

🏗️ Tech Stack (Python-Based)

🏗️ Tech Stack (Python-Based)

🔹 Backend

- **Python 3.10+**
- **FastAPI** (high-performance APIs)
- **Uvicorn** (ASGI server)
- **SQLAlchemy / Prisma ORM**
- **PostgreSQL / Supabase / SQLite**
- **JWT (python-jose / PyJWT)**
- **Passlib (bcrypt)**

🔹 File & PDF Handling

- **PyMuPDF (fitz)** or **PDF-Lib (via Python bindings)**
- **ReportLab** (optional)
- **Pillow** (signature images)

🔹 Frontend

- **React / Next.js**
- **Tailwind CSS**
- **react-pdf**
- **dnd-kit** (drag & drop)

📆 2-Week Build Plan (Python Stack)

📆 2-Week Build Plan (Python Stack)

✅ Week 1: Core Backend & Frontend Setup

Day 1: Project Setup

- Create GitHub repository
- Setup FastAPI project structure
- Configure PostgreSQL / Supabase
- Setup React + Tailwind frontend

Skills: FastAPI setup, project architecture

Day 2: Authentication System

- User model (id, name, email, password)
- Register & login APIs
- Password hashing with bcrypt
- JWT-based authentication middleware

Skills: Secure auth, token handling

Day 3: File Upload API

- Upload PDF using FastAPI `UploadFile`
- Save metadata in DB
- Store files locally or cloud

- Protect routes with JWT

Skills: File handling, secure APIs

Day 4: Document Listing & Preview

- Fetch user-specific documents
- Display dashboard
- Preview PDFs using react-pdf

Skills: REST APIs, frontend-backend integration

Day 5: Signature Schema

- Signature model (doc_id, user_id, x, y, page, status)
- Save signature positions
- Display placeholders on PDF

Skills: Coordinate mapping, schema relations

Day 6: Drag & Drop Signature UI

- Frontend overlay for signatures
- Save relative coordinates
- API: POST `/api/signatures`

Skills: UX logic, coordinate systems

Day 7: Buffer & Testing

- Postman API testing
- Debug integration issues

✅ Week 2: PDF Generation & Enterprise Features

Day 8: Generate Signed PDF

- Embed signature text/image using PyMuPDF
- Generate final PDF
- Store signed version

Skills: PDF processing, immutability

Day 9: Email & Public Links

- Generate tokenized public signing links
- Email signer securely

Skills: Token auth, email services

Day 10: Audit Trail

- Log user actions
- Store timestamp & IP
- API to fetch audit logs

Skills: Middleware, compliance logging

Day 11: Signature Status Flow

- Pending / Signed / Rejected
- Rejection reason handling

Skills: Business logic flows

Day 12: Dashboard Polish

- Filters by status
- Responsive Tailwind UI

Day 13: Deployment

- Backend: Render / Railway / Fly.io
- Frontend: Vercel / Netlify
- DB: Supabase / Neon

Day 14: Final Demo

- README documentation
- Demo video
- Sample credentials

📁 Python Project Folder Structure

📁 Python Project Folder Structure

```
/signature-app
|── /backend
|    ├── main.py
|    ├── /routers
|    ├── /models
|    ├── /schemas
|    ├── /services
|    ├── /middleware
|    ├── /utils
|    └── database.py
|
|── /frontend
|    ├── /src
|    |    ├── /components
|    |    ├── /pages
|    |    └── /utils
|
|── .env
|── README.md
```

🔎 API Endpoints

🔎 API Endpoints

Auth

```
POST /api/auth/register
POST /api/auth/login
```

Documents

```
POST /api/docs/upload
GET  /api/docs
GET  /api/docs/{id}
```

Signatures

```
POST /api/signatures
GET  /api/signatures/{docId}
POST /api/signatures/finalize
```

Audit

```
GET /api/audit/{docId}
```

🏢 Industry Value of This Project

🏢 Industry Value of This Project

Why This Project Matters

This project reflects **real enterprise-grade systems** used in:

- LegalTech
- HR platforms
- FinTech compliance tools
- Government portals
- Healthcare document systems

Skills Companies See Instantly

- Backend engineering depth (Python + FastAPI)
- Secure file handling
- PDF manipulation
- Audit and compliance logic
- SaaS product thinking

🏆 Why This Project Stands Out

Python-based DocuSign clones demonstrate backend maturity and enterprise thinking

Most portfolios lack:

- File workflows
- Audit trails

- PDF generation
- Security-first design

This project checks **all of them**.

Resources

Resources

📽️ YouTube Video Tutorials

🧠 FastAPI + Authentication : ▶ FastAPI with JWT auth tutorial

- Teaches how to add JWT authorization to a FastAPI backend using Python. Great for your auth system.

▶ Python FastAPI Tutorial (Part 10): Authentication - Registration and Login with JWT

- Step-by-step password hashing, JWT utilities, login & registration. Very close to what you need for your `/register` & `/login` endpoints.

📺 Additional FastAPI JWT tutorial:

- **FastAPI JWT Tutorial (auth + user login)** — https://youtu.be/0A_GCXBCNUQ
- **FastAPI JWT Setup Guide** — https://youtu.be/t1yDcoV446o
- **Protecting Routes with Authorization (Part 11)** — https://youtu.be/MY0TFMMm9B0

🎯 These videos are perfect for **authentication, JWT token creation, protected APIs, and securing routes**.

📄 Python PDF Handling (PyMuPDF) :
▶ Python PDF Handling with PyMuPDF 🇬🇧 🇺🇸 Session I [Part 1 of 14] #AT_PyMuPDF_EN

- A full multi-part video course on manipulating PDFs with Python — perfect for adding your signature logic and embedding text/images.

📺 Other PyMuPDF Python tutorials worth following:

- **PyMuPDF Coding Playlist** (PDF manipulation deep dive) — https://www.youtube.com/playlist?list=PLC1ikq_GXTvThHQb0ZeS16aI0NmZ69Q73
- **Automate PDF Form Filling with Python** (useful for signature placement logic) — ▶ Automate PDF Form Filling With Python | Python Automation
- **Extract text from PDF using Python (PyMuPDF)** — ▶ Extract Text From Pdf File Using Python || pyMuPdf || NLP

These help **read, modify, extract and embed content in PDFs** — critical for your final signed document generation.

📚 Documentation & Official Resources

🚀 FastAPI

- **FastAPI Official Docs** — https://fastapi.tiangolo.com/
  - Great for learning routing, dependency injection, security, file upload, and response models.
- **FastAPI OAuth2 & JWT Example (Security)** — https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/
  - Step-by-step guide to secure API creation with JWT.

🔐 JWT & Authentication Guides

- **FastAPI JWT Auth Tutorial (Text Guide)** — https://www.geeksforgeeks.org/python/login-registration-system-with-jwt-in-fastapi/
  - A textual tutorial guiding you through the authentication flow with code you can reuse.
- **Python JWT Tutorial (Core Concepts)** — Various Reddit and community discussions; search “FastAPI JWT examples” for insight on token handling and edge cases.

📜 PDF Manipulation

- **PyMuPDF Official Tutorial**
  → https://pymupdf.readthedocs.io/en/latest/tutorial.html
  - Covers everything from reading, modifying pages, embedding images and annotations — invaluable for building your signature rendering system.

🧩 Bonus Resources (Optional)

🛠️ Libraries & Practical Guidance

- **PyPDFForm (PDF Form Filling Library)** — Great for advanced PDF form generation and filling if you want to use templates.
- **TurboDocx Guide – Sign PDFs via API** — https://www.turbodocx.com/guides/sign-documents-with-python — useful if you want *external PDF signing APIs*.

📌 Quick Start Checklist (With Resources)

Feature	Resource Link
FastAPI Auth	FastAPI JWT video + OAuth2 docs
Secure Routes	FastAPI JWT protected routes video
File Upload & APIs	FastAPI docs on UploadFile
PDF Read/Write	PyMuPDF Tutorial + YouTube playlist
Signature Embedding	PyMuPDF video & docs
PDF Form Filling	PyPDFForm resource